Below is a **CDM v1.0** that stays **industry-agnostic + department-agnostic**, while still meeting your MVP needs: **map validated extraction into CDM**, preserve **end-to-end traceability**, support **entity resolution → golden records**, and publish **versioned datasets with dataset cards**.

Organization: The AI Institute Africa
System Name: Data Readiness System (DRS)
Document Type: CDM
Version: CDM v1.1
Prepared By: Wills
Date: February 2026
Project Timeline: 18 February 2026 – 11 April 2026
Methodology: V-Model + Hierarchical Model

CDM v1.0 design principles (non-negotiables)

1. **No hardcoded business entities** like Invoice/Receipt/Claim/Loan/etc.
2. **Everything is a "typed object"** with:
    - **taxonomy / classification label(s)** (e.g., `Invoice`, `Receipt`)
    - **evidence pointers** back to source pages/coordinates for each extracted value
3. **Golden records** are produced by **merging multiple candidate objects** using merge rules and decision logs (basic in MVP).
4. **Schema is versioned + extensible per tenant** without breaking core compatibility.

Core CDM entities (MVP-required set)

The MVP scope explicitly requires these canonical entities: **Document, Party, Event, Relationship, Dataset Version Metadata**.
This CDM implements those, and adds **supporting core objects** needed for "AI-ready / golden record" outcomes (evidence-traceable attributes, validation, and taxonomy).

---

1) Document

Represents a processed "evidence-backed" document.

Document fields

- `document_id` (PK)
- `tenant_id`
- `evidence_id` (FK to Evidence Object)

- `title` (optional)
- `document_date` (nullable)
- `source_system` / `source_type`
- `ingested_at`, `ingested_by`
- `document_type_hint` (from ingestion)
- `classification_label` (e.g., "Invoice") + `classification_confidence`
- `status` (e.g., ingested / processed / validated)

Why this matters: documents are *where traceability begins* (hash, immutable URI, provenance).

---

2) Party

A generic actor: **Person or Organization**. (Industry-agnostic.)

Party fields

- `party_id` (PK)
- `tenant_id`
- `party_kind` ENUM(`PERSON`,`ORG`)
- `display_name`
- `primary_identifier_id` (FK → Identifier, optional)
- `golden_record_id` (FK → Golden Record, optional)
- `created_at`, `updated_at`

---

3) Event

A generic "business occurrence": transaction/action/activity.

Event fields

- `event_id` (PK)
- `tenant_id`
- `event_time` (timestamp or date)
- `event_type_label` (taxonomy label, e.g., "Purchase", "Payment", "Submission")
- `event_confidence`
- `source_document_id` (FK → Document)
- `status` (candidate/validated/published)
- `created_at`, `updated_at`

---

4) Relationship

Connects Parties ↔ Events ↔ Documents ↔ Objects (generic linking).

Relationship fields

- `relationship_id` (PK)
- `tenant_id`
- `from_ref_type` (e.g., PARTY/EVENT/DOCUMENT/OBJECT)
- `from_ref_id`
- `to_ref_type`
- `to_ref_id`
- `relationship_type_label` (taxonomy label, e.g., “PAID_TO”, “OWNS”, “WORKS_FOR”)
- `confidence`
- `source_document_id` (FK → Document, optional)
- `created_at`

---

5) Dataset Version Metadata

Tracks published dataset versions + lineage.

DatasetVersion fields (minimum)

- `dataset_version_id` (PK)
- `tenant_id`
- `dataset_name`
- `dataset_version` (semver e.g., v1.0.0)
- `schema_version`
- `generated_at`
- `source_batches` / `lineage_evidence_batches`
- `validation_summary`
- `approvals` (if required)

Supporting CDM “must-have” objects for AI-ready / golden records

These do **not** introduce industry-specific entities; they provide the universal mechanics every org needs: identity, attributes, evidence traceability, validation provenance, and type labeling.

A) Object (generic business object / record)

This is the "thing" that may later be labeled Invoice/Receipt/Claim/etc.

Object fields

- `object_id` (PK)
- `tenant_id`
- `object_type_label` (taxonomy label: "Invoice", "Receipt", "Policy", etc.)
- `object_type_confidence`
- `source_document_id` (FK → Document)
- `status` (candidate/validated/resolved/published)
- `created_at`, `updated_at`

Note: If you prefer, you can treat **Event** as the object and keep Object optional, but in practice Object helps because many records are not pure "events" (e.g., "Policy", "Case file", "Asset register row").

B) Attribute (typed field/value, evidence-backed)

Stores the "golden-record-worthy" fields every company needs for trustworthy analytics: values with provenance.

Attribute fields

- `attribute_id` (PK)
- `tenant_id`
- `subject_ref_type` (OBJECT/PARTY/EVENT/DOCUMENT)
- `subject_ref_id`
- `attribute_key` (canonical key, e.g., `invoice_number`, `total_amount`, `address_line1`)
- `value_type` (string/number/date/boolean/json)
- `value_raw`
- `value_normalized` (optional; standardized formats)
- `unit` (optional)
- `currency` (optional)

- `confidence_score` (from extraction)
- `evidence_pointer` (FK → EvidencePointer)
- `validation_state` (pending/approved/rejected)
- `validated_by`, `validated_at`

C) EvidencePointer (traceability format)

Standard format that points to **page + bounding box coordinates** (and optionally text offsets).

EvidencePointer fields

- `pointer_id` (PK)
- `evidence_id`
- `page_number`
- `bbox_x1`, `bbox_y1`, `bbox_x2`, `bbox_y2`
- `snippet_text` (optional)
- `pointer_format_version` (so you can evolve it safely)

D) Identifier (universal entity resolution backbone)

Supports deterministic + fuzzy matching.

Identifier fields

- `identifier_id` (PK)
- `tenant_id`
- `subject_ref_type` (PARTY/OBJECT/ASSET/etc.)
- `subject_ref_id`
- `id_type_label` (taxonomy: NationalID, Passport, CompanyRegNo, InvoiceNo, AccountNo, etc.)
- `id_value`
- `issuing_authority` (optional)
- `is_verified` (boolean)
- `evidence_pointer` (FK → EvidencePointer, optional)

This is what lets “every company” create meaningful deduplication + golden records without hardcoding domain entities.

E) GoldenRecord (resolved master record)

Represents the **merged, best-known version** of a Party (and optionally Objects).

GoldenRecord fields

- `golden_record_id` (PK)
- `tenant_id`
- `subject_kind` (PARTY or OBJECT)
- `canonical_subject_id` (the surviving entity/object)
- `merged_from_ids` (list)
- `merge_rule_version`
- `match_score`
- `merge_decision_log_ref` (audit link)

F) SchemaRegistry (CDM schema + tenant extensions)

Versioned schema with migration support.

SchemaRegistry fields

- `schema_id`
- `schema_version`
- `schema_json`
- `created_by`
- `timestamp`
- `migration_scripts_ref` (optional)

How “AI-ready / golden record” requirements are satisfied (industry-agnostic)

To be AI-ready, every dataset needs:

- **Unique IDs + stable keys** (IDs, identifiers, hashes)
- **Standardized attributes** (normalized value + type + units)
- **Provenance** (who/when/from what evidence + pointers)

- **Validation status** (human approval trail)
- **Versioning** (schema + dataset versions)
- **Entity resolution outputs** (golden records + merge logs)

All of the above are covered by the combination of:

- Evidence immutability + metadata
- Evidence pointers format
- CDM mapping + schema versioning
- Golden record creation requirement
- Dataset publishing + dataset cards + versioning

Where taxonomy/classification lives (the "Invoice/Receipt/etc." label)

Implement a simple, extensible taxonomy:

Taxonomy tables

- `taxonomy_term(term_id, namespace, label, description, parent_term_id)`
- `classification(subject_ref_type, subject_ref_id, term_id, confidence, created_at, model_version)`

This is exactly aligned with your approach: **no Invoice entity**, but **records get labeled "Invoice"** via classification.

Your requested confirmation (explicit)

✅ **CONFIRMED:** The CDM does **not** hardcode Invoice/Receipt/etc. as entities, **but the system absolutely labels the Object/Document/Event as "Invoice/Receipt/etc." using classification/taxonomy metadata** (with confidence), and those labels flow into mapping, validation, publishing, and dataset cards.

If you want, next I can convert this into a **database-ready table list** (PostgreSQL-first) aligned to your "core tables" set (e.g., `cdm_entities`, `entity_links`, `dataset_versions`).